MGT-424: Machine Learning Methods in Econometrics
Homework 1

Instructions. Answer clearly and show the main steps. Use Python and submit both code and written answers. Standard packages such as `pandas`, `statsmodels`, and `sklearn.metrics` are allowed.

Question 1: Mortgage Default Prediction

A bank wants to predict mortgage default within 12 months. Let $Y_i = 1$ if household i defaults and $Y_i = 0$ otherwise. The bank models default probability as

$$\Pr(Y_i = 1 \mid X_i) = \Lambda(\beta_0 + \beta_1 \text{DTI}_i + \beta_2 \text{LTV}_i + \beta_3 \text{Score}_i + \beta_4 \text{UnempShock}_i + \beta_5 \text{PriorDelinq}_i)\,,$$

where $\Lambda(z) = \frac{e^z}{1+e^z}$, DTI is debt-to-income, LTV is loan-to-value, Score is credit score, `UnempShock`= 1 indicates a recent unemployment shock, and `PriorDelinq`= 1 indicates prior delinquency.

Suppose the estimated coefficients are

$$\hat{\beta}_0 = -8.00,\ \hat{\beta}_1 = 0.04,\ \hat{\beta}_2 = 0.03,\ \hat{\beta}_3 = -0.005,\ \hat{\beta}_4 = 0.90,\ \hat{\beta}_5 = 1.40.$$

(a) What is the target variable? For each regressor, state the expected sign and give a brief reason. Then give two practical reasons why logistic regression is preferable to a linear probability model here.

(b) For each borrower below, compute the linear index and predicted default probability. Then interpret $\hat{\beta}_5 = 1.40$ in terms of odds, where odds $= p/(1-p)$, and interpret the effect of a 50-point increase in credit score on the odds of default.

Borrower A: DTI = 28, LTV = 70, Score = 760, UnempShock = 0, PriorDelinq = 0.

Borrower B: DTI = 75, LTV = 105, Score = 540, UnempShock = 1, PriorDelinq = 1.

(c) The bank considers two classification thresholds. On a test set of 200 mortgages, the results are:

Threshold	TP	FP	TN	FN
0.50	18	7	153	22
0.30	30	20	140	10

This table is a confusion matrix: TP = true positives, FP = false positives, TN = true negatives, and FN = false negatives. Use accuracy = $(TP+TN)/N$, precision = $TP/(TP+FP)$, and recall = $TP/(TP+FN)$. Which threshold would you recommend if the bank especially wants to avoid missing true defaulters? Briefly explain why the coefficient on UnempShock is not automatically causal[1]. Also, if local house-price growth reduces default and is negatively correlated with LTV, what bias would omitting it create in the estimated coefficient on LTV?

(d) Empirical implementation. Use the files `mortgage_train.csv` and `mortgage_test.csv` in the `datasets/` folder. They contain the outcome and the five predictors used in this question. Using Python, do the following:

(i) First estimate a logistic regression of `default` on the five predictors using your own gradient descent implementation on the training data. Report the estimated coefficients and compare their signs with your answer in part (a). If any sign differs from your prior expectation, briefly explain why that can happen in a finite sample. Then estimate the same model using `statsmodels.api.Logit` and briefly compare the two sets of results.

(ii) Using your fitted logistic regression, compute predicted probabilities for all observations in the test data. Using thresholds 0.50 and 0.30, classify each observation and report the confusion matrix, accuracy, precision, and recall for each threshold.

(iii) Compare the two thresholds. Which one would you recommend if false negatives are more costly than false positives?

(iv) Estimate a linear probability model using OLS on the same training data and briefly compare it with logistic regression.

Question 2: Count Modeling and Generalized Linear Models

Many real-world applications involve predicting the number of times an event occurs within a fixed period, such as customer arrivals, equipment failures, insurance claims, or bicycle rentals. These outcomes are non-negative integers and typically exhibit skewed distributions, making standard Gaussian-based linear regression inappropriate.

In this exercise, you will analyze count data using both linear regression and a generalized linear model (GLM). The goal is to understand how modeling assumptions affect estimation and prediction when the response variable is non-Gaussian.

The file `daily-bike-share.csv` in the `datasets/` folder records daily bicycle rental counts together with weather and calendar information. Predicting demand for shared bicycles is an important practical problem, as operators must allocate bicycles, plan maintenance, and manage staffing based on expected usage.

Preliminary Distribution Analysis

Before fitting any model, you will examine the distribution of the response variable. In particular, you will compute the Kolmogorov–Smirnov (KS) statistic to assess whether the data are consistent

[1] Here, *causal* means a cause-and-effect interpretation: changing `UnempShock` would itself change default risk, rather than merely being correlated with it.

with a normal distribution. This step highlights the limitations of Gaussian models for count outcomes.

Models

You will compare two approaches:

- A linear regression model estimated under Gaussian assumptions
- A Poisson generalized linear model for count data

The Poisson GLM uses a log link function, ensuring non-negative predictions and modeling the conditional mean as an exponential function of the covariates.

Implementation

To develop a deeper understanding of these methods, you must implement the estimation procedures directly using basic numerical tools. High-level libraries that automatically fit regression or GLM models should not be used for the main implementation.

After completing your implementation, you will compare your results with those obtained using standard statistical libraries. This step serves to verify correctness and illustrate potential numerical differences between implementations.

Tasks

Complete the assignment by following the instructions in the accompanying Python notebook. The notebook specifies all required steps, including distribution testing, model implementation, estimation, comparison, and interpretation.

Question 3: Sentiment Classification with Naive Bayes

An *n-gram* is a contiguous sequence of n items (typically words or tokens) taken from a text. N-grams are commonly used in natural language processing to analyze language patterns and to build language models that estimate the likelihood of word sequences.

For example, in the sentence "I love natural language processing":

- Unigram (1-gram): I, love, natural, language, processing
- Bigram (2-gram): I love, love natural, natural language, language processing
- Trigram (3-gram): I love natural, love natural language, natural language processing

N-grams can also be defined as a **range** rather than a single value of n. For instance, a range such as $n = 1$–3 means that the model considers unigrams, bigrams, and trigrams together. Using a range of n-grams allows models to capture both short and slightly longer contextual relationships in text.

(a) Why it is better to use n-grams instead of only using unigrams? Explain with an example.

(b) Provide an example showing how using n-gram features in a Naive Bayes model can violate the naive independence assumption. Then discuss why Naive Bayes still works well in practice despite this violation.

(c) Why do higher-order n-grams (e.g., 4-grams, 5-grams) often suffer from *data sparsity*?

(d) How can smoothing help with data sparsity?

(e) In this part you will implement a sentiment classification system using Python and the `scikit-learn` library. The goal is to evaluate how different *n-gram ranges* affect the performance and runtime of a Naive Bayes classifier.

The experiments in this assignment use the Financial PhraseBank dataset. Students are provided with the files `datasets/FPB_train.csv` and `datasets/FPB_test.csv`, which are derived from this dataset.

You must use the following Python libraries in your implementation:

- `numpy`
- `pandas`
- `time`
- `matplotlib.pyplot`
- `seaborn`
- `sklearn.feature_extraction.text.CountVectorizer`
- `sklearn.naive_bayes.MultinomialNB`
- `sklearn.metrics.accuracy_score`

You must implement the following steps:

(i) Read the train and test dataset files using `pandas`.

(ii) Implement a loop to test different **n-gram ranges**. The starting value should vary from 1 to 7 and the ending value should vary from the starting value up to 7.

For each n-gram range, convert the text into feature vectors using `CountVectorizer`. Train a `MultinomialNB` classifier with parameter `alpha` equal to one using the training dataset. Then predict the labels for the test set and compute the classification accuracy using `accuracy_score` and measure the runtime of the experiment using `time`.

(iii) Store the classification accuracies and runtimes in two two-dimensional matrices indexed by the n-gram range start and end points. Then generate two heatmaps from these matrices using `seaborn` and display the results using `matplotlib`.

Your program should generate two visualizations:

- Heatmap showing classification accuracy for each n-gram range
- Heatmap showing runtime for each experiment

(f) Discuss whether increasing the range of n-grams always improves model predictions. What effect does increasing the n-gram range have on the runtime of the model?